

Type Systems

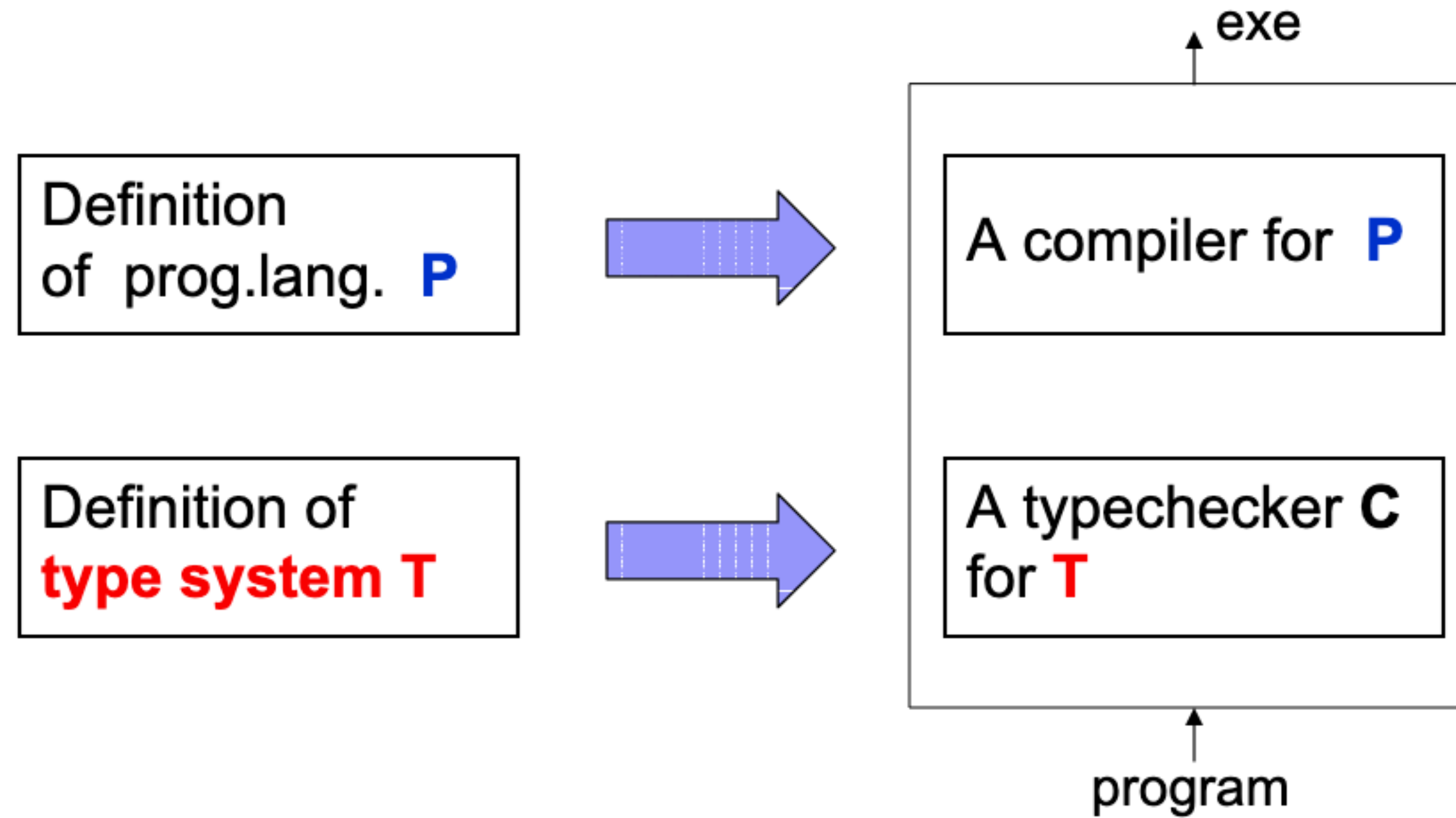
Takes material from

EPFL type systems course: <http://lampwww.epfl.ch/teaching/typeSystems/2004>

and KATHOLIEKE UNIVERSITEIT LEUVEN Type systems course

Why and What (are Types)

Type Systems for Programming Languages



→ is (**P**, **T**) type sound?

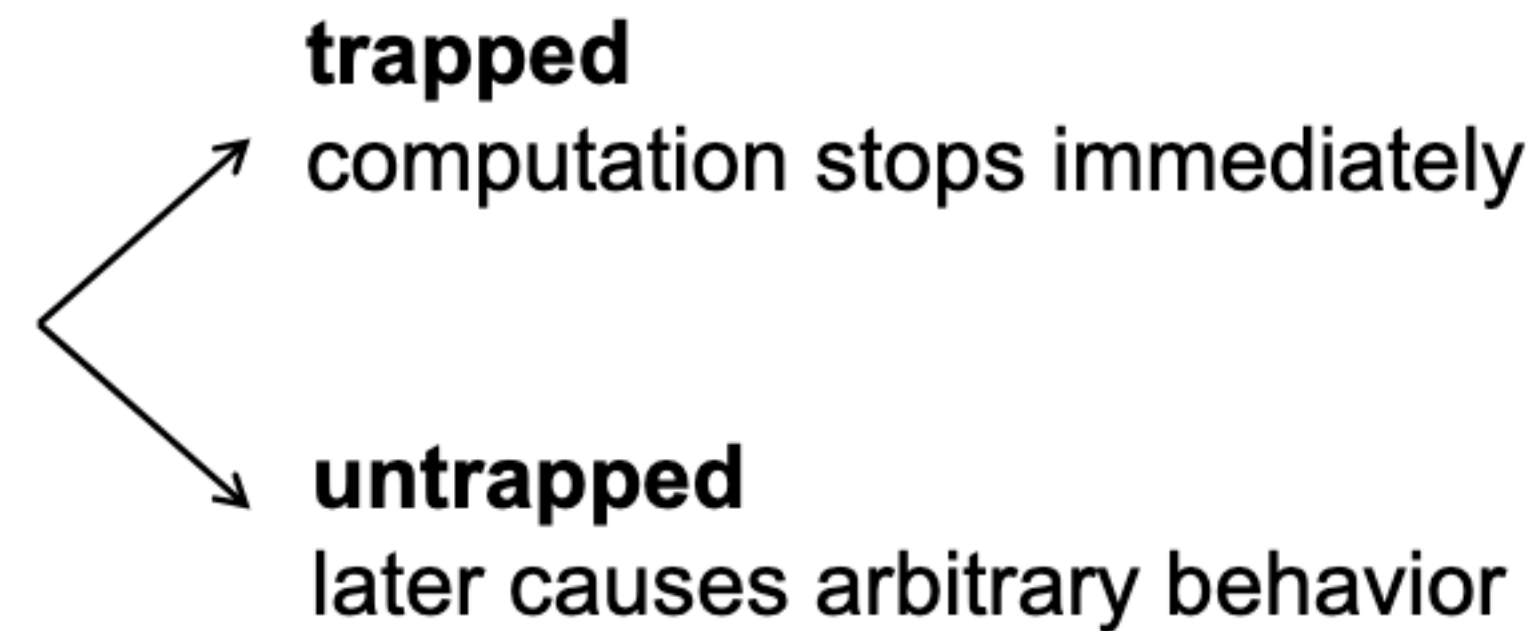
→ is **T** decidable?

→ does **C** correctly implement **T**?

What are they good for?

- To prevent execution errors.

Execution Errors



examples:

- division by zero
- accessing an illegal addr.
- jump to a wrong addr.
- accessing past the end of an array

A program is **SAFE** if it does not have untrapped errors.

A PL is **SAFE** if all its programs are safe.

What is a Type, in PL context, informally

A **type** is an upper bound of the **range of values** that a **program variable** can assume during execution.

e.g. if x has **type Boolean**, then in all runs it should only take one of the values **true / false**.

→ $\text{not}(x)$ has a meaning in every run

PLs in which variables can be given nontrivial types are called **TYPED languages**.

safe/unsafe and typed/untyped

	typed	untyped
safe	ML, Java	LISP
unsafe	C	Assembler

safety $\Rightarrow$ integrity of run-time structures

$\Rightarrow$ enables **garbage collection**



We will study Types on a simplified yet powerful Language: **saves code size / develop. time**

Arithmetic expr, Lambda Calculus (price: performance)

Arithmetic Expression Language: Syntax

`t ::=`

`true`
`false`
`if t then t else t`
`0`
`succ t`
`pred t`
`iszero t`

`v ::=`

`true`
`false`
`nv`

`nv ::=`

`0`
`succ nv`

terms

constant true
constant false
conditional
constant zero
successor
predecessor
zero test

values

true value
false value
numeric value

numeric values

zero value
successor value

Arithmetic Expression Language: Semantics

$$\frac{t_1 \longrightarrow t'_1}{\text{succ } t_1 \longrightarrow \text{succ } t'_1} \quad (\text{E-SUCC})$$

$$\text{if true then } t_2 \text{ else } t_3 \longrightarrow t_2 \quad (\text{E-IFTRUE})$$

$$\text{pred } 0 \longrightarrow 0 \quad (\text{E-PREDZERO})$$

$$\text{if false then } t_2 \text{ else } t_3 \longrightarrow t_3 \quad (\text{E-IFFALSE})$$

$$\text{pred (succ } nv_1) \longrightarrow nv_1 \quad (\text{E-PREDSUCC})$$

$$\frac{t_1 \longrightarrow t'_1}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \longrightarrow \text{if } t'_1 \text{ then } t_2 \text{ else } t_3} \quad (\text{E-IF})$$

$$\frac{t_1 \longrightarrow t'_1}{\text{pred } t_1 \longrightarrow \text{pred } t'_1} \quad (\text{E-PRED})$$

$$\text{iszero } 0 \longrightarrow \text{true} \quad (\text{E-ISZEROZERO})$$

$$\text{iszero (succ } nv_1) \longrightarrow \text{false} \quad (\text{E-ISZEROSUCC})$$

$$\frac{t_1 \longrightarrow t'_1}{\text{iszero } t_1 \longrightarrow \text{iszero } t'_1} \quad (\text{E-ISZERO})$$

Example Runtime Errors

- The goal of a type system is to avoid “getting stuck”
 - E.g. never reach: `succ true`
- The type system will do this by
 - Statically classifying terms according to “the type of value” they compute
 - Ensuring that only the right “types” of values are used in each part of the program
 - E.g. only a boolean is used as condition in a test

Typing Relation

- We will distinguish two “types of values” (i.e. two *types*): natural numbers and Booleans

$T ::=$

Bool

Nat

types

type of booleans

type of numbers

- And we will define a relation between terms and types
 - $t:T$ -- read as “ t is of type T ” or “ t has type T ”
 - This relation is defined inductively, using inference rules

Typing Rules for Typing Relations

$\text{true} : \text{Bool}$

(T-TRUE)

$\text{false} : \text{Bool}$

(T-FALSE)

$$\frac{t_1 : \text{Bool} \quad t_2 : T \quad t_3 : T}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 : T}$$

(T-IF)

$0 : \text{Nat}$

(T-ZERO)

$t_1 : \text{Nat}$

$\text{succ } t_1 : \text{Nat}$

(T-SUCC)

$t_1 : \text{Nat}$

$\text{pred } t_1 : \text{Nat}$

(T-PRED)

$t_1 : \text{Nat}$

$\text{iszero } t_1 : \text{Bool}$

(T-ISZERO)

A correct program will have a valid typing derivation

$$\frac{\frac{\frac{}{0 : \text{Nat}} \text{T-ZERO} \quad \frac{}{\text{iszero } 0 : \text{Bool}} \text{T-ISZERO}}{\text{if iszero } 0 \text{ then } 0 \text{ else pred } 0 : \text{Nat}} \text{T-IF} \quad \frac{\frac{}{0 : \text{Nat}} \text{T-ZERO} \quad \frac{\frac{}{0 : \text{Nat}} \text{T-PRED}}{\text{pred } 0 : \text{Nat}} \text{T-IF}}{\text{if iszero } 0 \text{ then } 0 \text{ else pred } 0 : \text{Nat}} \text{T-IF}$$

Similar to Proofs in FO/Predicate Logic Formulas

Type Soundness

- The type system is designed to avoid run-time errors
 - Now, we should prove it does this correctly: A well-typed program does not get stuck
- Note that the converse is not generally true
 - Programs that do not get stuck are not necessarily well-typed
 - E.g. if true then 1 else succ false
 - Typically unavoidable because of the static nature of typing

Proving Type Soundness

Safety = Progress + Preservation

The safety (or soundness) of this type system can be expressed by two properties:

1. *Progress*: A well-typed term is not stuck

If $t : T$, then either t is a value or else $t \longrightarrow t'$ for some t' .

2. *Preservation*: Types are preserved by one-step evaluation

If $t : T$ and $t \longrightarrow t'$, then $t' : T$.

Untyped Lambda Calculus : Syntax

$t ::=$

x

$\lambda x. t$

$t \ t$

terms

variable

abstraction

application

- ▶ Application associates to the left

E.g., $t \ u \ v$ means $(t \ u) \ v$, not $t \ (u \ v)$

- ▶ Bodies of λ - abstractions extend as far to the right as possible

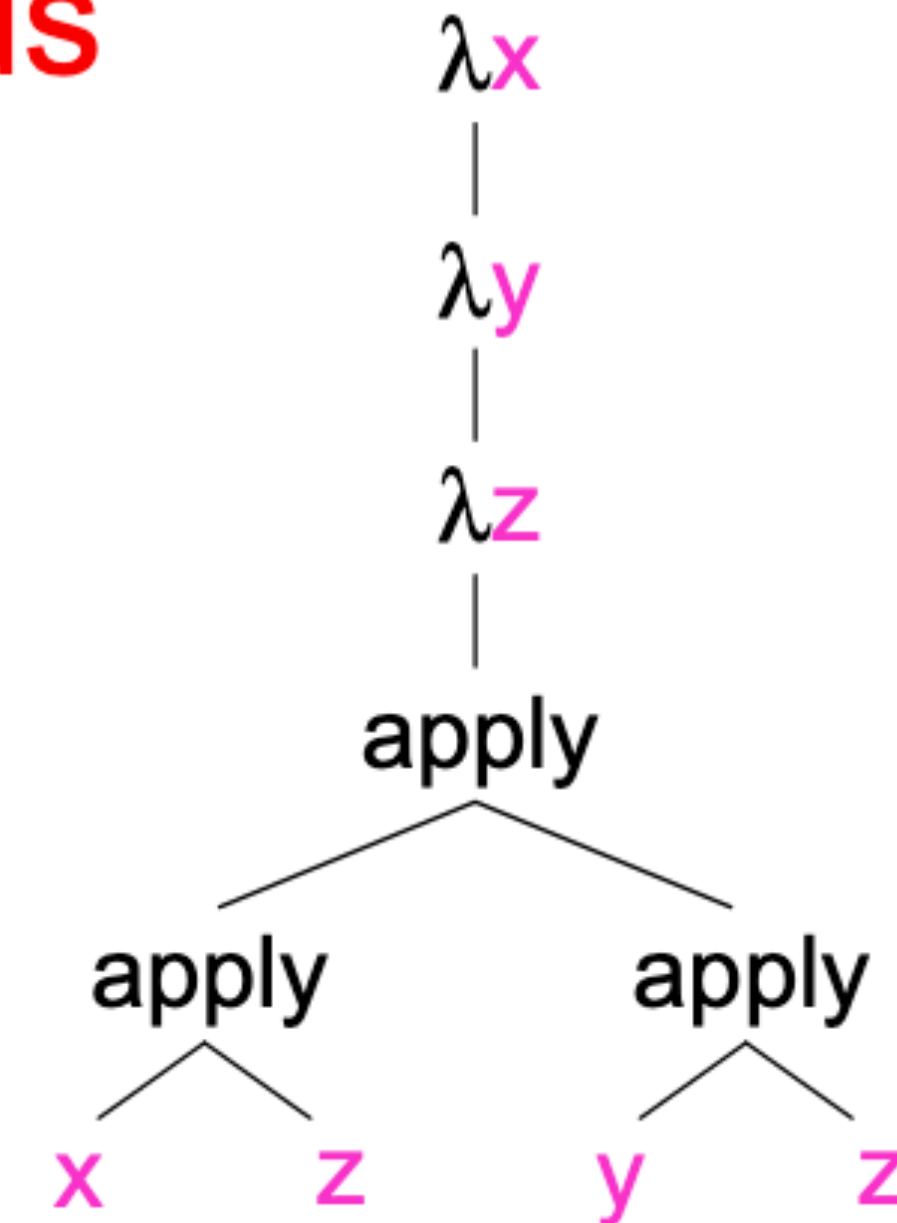
E.g., $\lambda x. \ \lambda y. \ x \ y$ means $\lambda x. \ (\lambda y. \ x \ y)$, not $\lambda x. \ (\lambda y. \ x) \ y$

2. Syntax of the Lambda Calculus

Example:

$$\begin{aligned} & \lambda x. \lambda y. \lambda z. x z (y z) \\ &= \lambda x. (\lambda y. \lambda z. x z (y z)) \\ &= \lambda x. (\lambda y. (\lambda z. (x z (y z)))) \\ &= \lambda x. (\lambda y. (\lambda z. ((x z) (y z)))) \end{aligned}$$

“surface syntax”



“abstract syntax”

Conventions (to save parenthesis)

application is **left** associative: $x y z = (x y) z \neq x (y z)$

scope of **abstraction** extends **as far to the right as possible**:

$$\lambda x. x y = \lambda x. (x y) \neq (\lambda x. x) y$$

Scope and free variables

- In the term $\lambda x.t$, the variable x is **bound** in t
 t is the **scope** of the binding
- A variable is **free** if it is not bound by any enclosing abstraction
- A term without free variables is called a **closed term**

Untyped Lambda Calculus : Operation Semantics

$t ::=$	<i>terms</i>
x	<i>variable</i>
$\lambda x. t$	<i>abstraction</i>
$t \ t$	<i>application</i>
$v ::=$	<i>values</i>
$\lambda x. t$	<i>abstraction value</i>

Computation rule:

$$(\lambda x. t_{12}) \ v_2 \longrightarrow [x \mapsto v_2] t_{12} \quad (\text{E-APPABS})$$

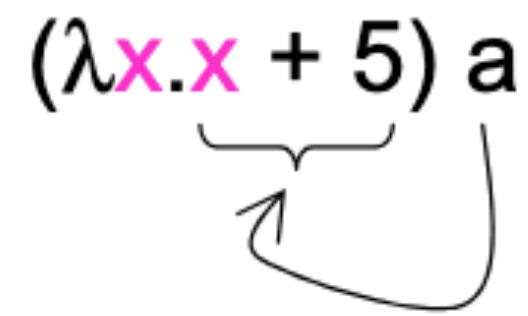
Congruence rules:

$$\frac{t_1 \longrightarrow t'_1}{t_1 \ t_2 \longrightarrow t'_1 \ t_2} \quad (\text{E-APP1})$$

$$\frac{t_2 \longrightarrow t'_2}{v_1 \ t_2 \longrightarrow v_1 \ t'_2} \quad (\text{E-APP2})$$

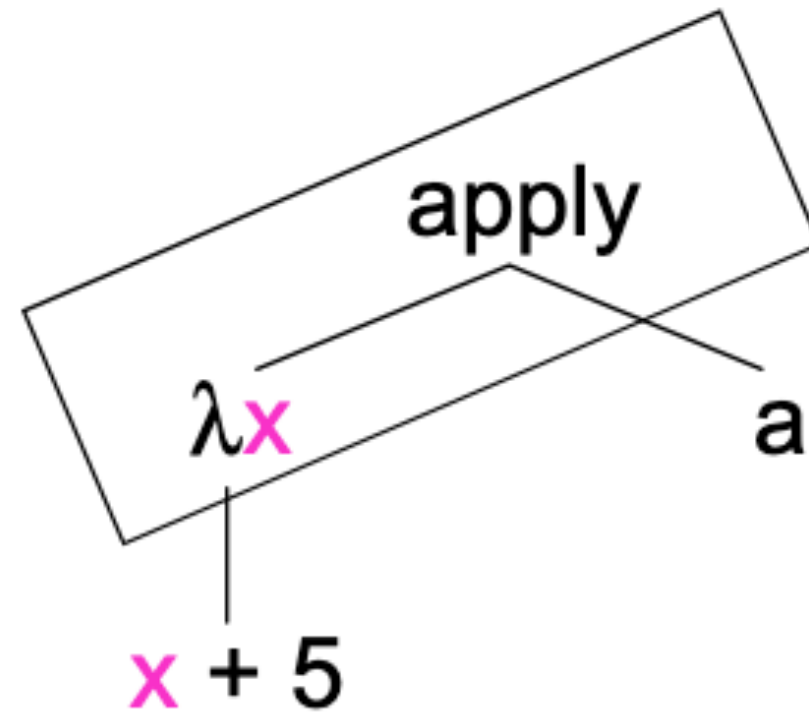
2. Semantics of the Lambda Calculus

$(\lambda x. x + 5) a$



A bracket under 'x + 5' in the expression $(\lambda x. x + 5) a$ has an arrow pointing from it to the 'a' argument, indicating the substitution process.

SUBSTITUTE a for x in $x + 5$



redex (REDucible EXpression): $(\lambda x. t) t_1$

can be reduced to (evaluates to):

$$\xrightarrow{\beta\text{-reduction}} [x \rightarrow a] x + 5 = a + 5$$

To compute in Lambda Calculus, ALL you do is **SUBSTITUTE!!**

Simply Typed Lambda Calculus : Syntax

$t ::=$

x

$\lambda x.t$

$t\ t$

true

false

$\text{if } t \text{ then } t \text{ else } t$

$v ::=$

$\lambda x.t$

true

false

terms

variable

abstraction

application

constant true

constant false

conditional

values

abstraction value

true value

false value

Simple Type Systems

Typing an expression language

- Recap: arithmetic expressions
 - Syntax, semantics
 - Run-time errors
- The Typing Relation
- Type soundness
- Wrap-up

Recap: syntax

t ::=

true
false
if t then t else t
0
succ t
pred t
iszero t

terms

constant true
constant false
conditional
constant zero
successor
predecessor
zero test

v ::=

true
false
nv

values

true value
false value
numeric value

nv ::=

0
succ nv

numeric values

zero value
successor value

Recap: semantics (1)

$\text{if true then } t_2 \text{ else } t_3 \longrightarrow t_2 \quad (\text{E-IFTRUE})$

$\text{if false then } t_2 \text{ else } t_3 \longrightarrow t_3 \quad (\text{E-IFFALSE})$

$$\frac{t_1 \longrightarrow t'_1}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \longrightarrow \text{if } t'_1 \text{ then } t_2 \text{ else } t_3} \quad (\text{E-IF})$$

Recap: semantics (2)

$$\frac{t_1 \longrightarrow t'_1}{\text{succ } t_1 \longrightarrow \text{succ } t'_1} \quad (\text{E-SUCC})$$

$$\text{pred } 0 \longrightarrow 0 \quad (\text{E-PREDZERO})$$

$$\text{pred } (\text{succ } nv_1) \longrightarrow nv_1 \quad (\text{E-PREDSUCC})$$

$$\frac{t_1 \longrightarrow t'_1}{\text{pred } t_1 \longrightarrow \text{pred } t'_1} \quad (\text{E-PRED})$$

$$\text{iszero } 0 \longrightarrow \text{true} \quad (\text{E-ISZEROZERO})$$

$$\text{iszero } (\text{succ } nv_1) \longrightarrow \text{false} \quad (\text{E-ISZEROSUCC})$$

$$\frac{t_1 \longrightarrow t'_1}{\text{iszero } t_1 \longrightarrow \text{iszero } t'_1} \quad (\text{E-ISZERO})$$

Run-time errors

- The goal of a type system is to avoid “getting stuck”
 - E.g. never reach: succ true
- The type system will do this by
 - Statically classifying terms according to “the type of value” they compute
 - Ensuring that only the right “types” of values are used in each part of the program
 - E.g. only a boolean is used as condition in a test

The Typing Relation

- We will distinguish two “types of values” (i.e. two *types*): natural numbers and Booleans

$T ::=$	<i>types</i>
Bool	<i>type of booleans</i>
Nat	<i>type of numbers</i>

- And we will define a relation between terms and types
 - $t:T$ *-- read as “t is of type T” or “t has type T”*
 - This relation is defined inductively, using inference rules

The Typing Rules

$\text{true} : \text{Bool}$ (T-TRUE)

$\text{false} : \text{Bool}$ (T-FALSE)

$$\frac{t_1 : \text{Bool} \quad t_2 : T \quad t_3 : T}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 : T}$$
 (T-IF)

$0 : \text{Nat}$ (T-ZERO)

$$\frac{t_1 : \text{Nat}}{\text{succ } t_1 : \text{Nat}}$$
 (T-SUCC)

$$\frac{t_1 : \text{Nat}}{\text{pred } t_1 : \text{Nat}}$$
 (T-PRED)

$$\frac{t_1 : \text{Nat}}{\text{iszero } t_1 : \text{Bool}}$$
 (T-ISZERO)

Example of a type derivation tree

$$\frac{\frac{\frac{}{0 : \text{Nat}} \text{T-ZERO}}{\text{iszero } 0 : \text{Bool}} \text{T-ISZERO} \quad \frac{\frac{}{0 : \text{Nat}} \text{T-ZERO}}{\text{pred } 0 : \text{Nat}} \text{T-PRED}}{\text{if iszero } 0 \text{ then } 0 \text{ else pred } 0 : \text{Nat}} \text{T-IF}$$

A *subderivation* is a subtree of a derivation.

Type Soundness

- The type system is designed to avoid run-time errors
 - Now, we should prove it does this correctly: A well-typed program does not get stuck
- Note that the converse is not generally true
 - Programs that do not get stuck are not necessarily well-typed
 - E.g. if true then 1 else succ false
 - Typically unavoidable because of the static nature of typing

Proving Type Soundness

Safety = Progress + Preservation

The safety (or soundness) of this type system can be expressed by two properties:

1. *Progress*: A well-typed term is not stuck

If $t : T$, then either t is a value or else $t \longrightarrow t'$ for some t' .

2. *Preservation*: Types are preserved by one-step evaluation

If $t : T$ and $t \longrightarrow t'$, then $t' : T$.

Lemma: Inversion

Lemma:

1. If $\text{true} : R$, then $R = \text{Bool}$.
2. If $\text{false} : R$, then $R = \text{Bool}$.
3. If $\text{if } t_1 \text{ then } t_2 \text{ else } t_3 : R$, then $t_1 : \text{Bool}$, $t_2 : R$, and $t_3 : R$.
4. If $0 : R$, then $R = \text{Nat}$.
5. If $\text{succ } t_1 : R$, then $R = \text{Nat}$ and $t_1 : \text{Nat}$.
6. If $\text{pred } t_1 : R$, then $R = \text{Nat}$ and $t_1 : \text{Nat}$.
7. If $\text{iszero } t_1 : R$, then $R = \text{Bool}$ and $t_1 : \text{Nat}$.

Proof: Immediate from the definition of the typing relation.

Progress and Preservation

- Canonical Forms Lemma:

Lemma:

1. If v is a value of type `Bool`, then v is either `true` or `false`.
2. If v is a value of type `Nat`, then v is a numeric value.

- Progress:

Theorem: Suppose t is a well-typed term (that is, $t : T$ for some T). Then either t is a value or else there is some t' with $t \longrightarrow t'$.

- Preservation:

Theorem: If $t : T$ and $t \longrightarrow t'$, then $t' : T$.

Wrap-up

- “Type-checking” consists of a static check of the program, such that runtime errors are avoided
- Proving a type system sound:
 - Generalize “well-typed” to program states
 - Is trivial here
 - Prove that well-typed program states do not get stuck (Progress)
 - Prove that “being well-typed” is preserved by evaluation steps (Preservation)

Course Overview

- Introduction
- Formal models of programming languages
- Simple type systems
 - Typing an expression language
 - Simply typed lambda calculus
 - Typing simple programming language features: records, variants, lists, references, exceptions, ...
- Subtyping
- Conclusion

Simply Typed Lambda Calculus

- We will study a series of type systems for the lambda calculus.
- This first one is more or less the “simplest one possible”
 - We need at least a “type of functions”: $\rightarrow$
 - But in order to distinguish higher-order functions, we need more than just a “type of functions”

$T ::=$	<i>types</i>
Bool	<i>type of booleans</i>
$T \rightarrow T$	<i>types of functions</i>